

pandas-lab

October 3, 2025

```
[1]: import pandas as pd
import numpy as np
```

```
[2]: data = pd.read_csv('Pfizer_1.csv')
data.head()
```

```
[2]:
```

	Date	Drug_Name	Parameter	1:30:00	2:30:00	\
0	15-10-2020	diltiazem hydrochloride	Temperature	23.0	22.0	
1	15-10-2020	diltiazem hydrochloride	Pressure	12.0	13.0	
2	15-10-2020	docetaxel injection	Temperature	NaN	17.0	
3	15-10-2020	docetaxel injection	Pressure	NaN	22.0	
4	15-10-2020	ketamine hydrochloride	Temperature	24.0	NaN	

	3:30:00	4:30:00	5:30:00	6:30:00	7:30:00	8:30:00	9:30:00	10:30:00	\
0	NaN	21.0	21.0	22	23.0	21.0	22.0	20	
1	NaN	11.0	13.0	14	16.0	16.0	24.0	18	
2	18.0	NaN	17.0	18	NaN	NaN	23.0	23	
3	22.0	NaN	22.0	23	NaN	NaN	27.0	26	
4	NaN	27.0	NaN	26	25.0	24.0	23.0	22	

	11:30:00	12:30:00
0	20.0	21
1	19.0	20
2	25.0	25
3	29.0	28
4	21.0	20

```
[3]: pd.melt(data, id_vars=['Date', 'Parameter', 'Drug_Name'])
```

```
[3]:
```

	Date	Parameter	Drug_Name	variable	value
0	15-10-2020	Temperature	diltiazem hydrochloride	1:30:00	23.0
1	15-10-2020	Pressure	diltiazem hydrochloride	1:30:00	12.0
2	15-10-2020	Temperature	docetaxel injection	1:30:00	NaN
3	15-10-2020	Pressure	docetaxel injection	1:30:00	NaN
4	15-10-2020	Temperature	ketamine hydrochloride	1:30:00	24.0
..	...	...	...	...	...
211	17-10-2020	Pressure	diltiazem hydrochloride	12:30:00	14.0

212	17-10-2020	Temperature	docetaxel injection	12:30:00	23.0
213	17-10-2020	Pressure	docetaxel injection	12:30:00	28.0
214	17-10-2020	Temperature	ketamine hydrochloride	12:30:00	24.0
215	17-10-2020	Pressure	ketamine hydrochloride	12:30:00	15.0

[216 rows x 5 columns]

```
[5]: data_melt = pd.melt(data, id_vars=['Date', 'Parameter', 'Drug_Name'],
    var_name='time',
    value_name='readings')
```

```
[8]: data_melt.head()
```

```
[8]:
```

	Date	Parameter	Drug_Name	time	readings
0	15-10-2020	Temperature	diltiazem hydrochloride	1:30:00	23.0
1	15-10-2020	Pressure	diltiazem hydrochloride	1:30:00	12.0
2	15-10-2020	Temperature	docetaxel injection	1:30:00	NaN
3	15-10-2020	Pressure	docetaxel injection	1:30:00	NaN
4	15-10-2020	Temperature	ketamine hydrochloride	1:30:00	24.0

```
[16]: data_tidy = data_melt.pivot(index=['Date', 'Drug_Name', 'time'],
    columns='Parameter',
    values='readings')
```

```
[18]: data_tidy
```

```
[18]:
```

Parameter		Pressure	Temperature
Date	Drug_Name	time	
15-10-2020	diltiazem hydrochloride	10:30:00	18.0
		11:30:00	19.0
		12:30:00	20.0
		1:30:00	12.0
		2:30:00	13.0
...			
17-10-2020	ketamine hydrochloride	5:30:00	11.0
		6:30:00	12.0
		7:30:00	12.0
		8:30:00	11.0
		9:30:00	12.0

[108 rows x 2 columns]

```
[19]: data_tidy = data_tidy.reset_index()
```

```
[20]: data_tidy.head()
```

```
[20]: Parameter      Date      Drug_Name      time      Pressure  \
0      15-10-2020  diltiazem hydrochloride  10:30:00      18.0
1      15-10-2020  diltiazem hydrochloride  11:30:00      19.0
2      15-10-2020  diltiazem hydrochloride  12:30:00      20.0
3      15-10-2020  diltiazem hydrochloride   1:30:00      12.0
4      15-10-2020  diltiazem hydrochloride   2:30:00      13.0
```

```
Parameter  Temperature
0          20.0
1          20.0
2          21.0
3          23.0
4          22.0
```

```
[21]: # String methods
```

```
[22]: data_tidy['Drug_Name']
```

```
[22]: 0      diltiazem hydrochloride
1      diltiazem hydrochloride
2      diltiazem hydrochloride
3      diltiazem hydrochloride
4      diltiazem hydrochloride
...
103     ketamine hydrochloride
104     ketamine hydrochloride
105     ketamine hydrochloride
106     ketamine hydrochloride
107     ketamine hydrochloride
Name: Drug_Name, Length: 108, dtype: object
```

```
[24]: type(data_tidy['Drug_Name'])
```

```
[24]: pandas.core.series.Series
```

```
[25]: "rahul".upper()
```

```
[25]: 'RAHUL'
```

```
[23]: data_tidy['Drug_Name'].upper()
```

```
-----
AttributeError                                Traceback (most recent call last)
/var/folders/t5/yhjrjs907zfp250jyxtw54m0000gn/T/ipykernel_58635/2104651725.py
  ↪in <module>
----> 1 data_tidy['Drug_Name'].upper()
```

```
~/opt/anaconda3/lib/python3.9/site-packages/pandas/core/generic.py in
-> __getattr__(self, name)
5485         ):
5486             return self[name]
-> 5487         return object.__getattribute__(self, name)
5488
5489     def __setattr__(self, name: str, value) -> None:

AttributeError: 'Series' object has no attribute 'upper'
```

```
[26]: data_tidy['Drug_Name'].str.upper()
```

```
[26]: 0      DILTIAZEM HYDROCHLORIDE
      1      DILTIAZEM HYDROCHLORIDE
      2      DILTIAZEM HYDROCHLORIDE
      3      DILTIAZEM HYDROCHLORIDE
      4      DILTIAZEM HYDROCHLORIDE
      ...
     103      KETAMINE HYDROCHLORIDE
     104      KETAMINE HYDROCHLORIDE
     105      KETAMINE HYDROCHLORIDE
     106      KETAMINE HYDROCHLORIDE
     107      KETAMINE HYDROCHLORIDE
      Name: Drug_Name, Length: 108, dtype: object
```

```
[29]: # HW: Do the above changes inplace
```

```
[33]: data_tidy.head()
```

```
[33]: Parameter      Date      Drug_Name      time  Pressure  \
0      15-10-2020  diltiazem hydrochloride  10:30:00      18.0
1      15-10-2020  diltiazem hydrochloride  11:30:00      19.0
2      15-10-2020  diltiazem hydrochloride  12:30:00      20.0
3      15-10-2020  diltiazem hydrochloride   1:30:00      12.0
4      15-10-2020  diltiazem hydrochloride   2:30:00      13.0

Parameter  Temperature
0          20.0
1          20.0
2          21.0
3          23.0
4          22.0
```

```
[34]: # check if your drug have hydrochloride in it or not?
```

```
[35]: data_tidy['Drug_Name'].str.contains('hydrochloride')
```

```
[35]: 0      True
      1      True
      2      True
      3      True
      4      True
      ...
     103     True
     104     True
     105     True
     106     True
     107     True
      Name: Drug_Name, Length: 108, dtype: bool
```

```
[36]: data_tidy[data_tidy['Drug_Name'].str.contains('hydrochloride')]
```

```
[36]: Parameter      Date      Drug_Name      time  Pressure  \
0      15-10-2020  diltiazem hydrochloride  10:30:00      18.0
1      15-10-2020  diltiazem hydrochloride  11:30:00      19.0
2      15-10-2020  diltiazem hydrochloride  12:30:00      20.0
3      15-10-2020  diltiazem hydrochloride   1:30:00      12.0
4      15-10-2020  diltiazem hydrochloride   2:30:00      13.0
..      ...
103     17-10-2020   ketamine hydrochloride   5:30:00      11.0
104     17-10-2020   ketamine hydrochloride   6:30:00      12.0
105     17-10-2020   ketamine hydrochloride   7:30:00      12.0
106     17-10-2020   ketamine hydrochloride   8:30:00      11.0
107     17-10-2020   ketamine hydrochloride   9:30:00      12.0
```

```
Parameter  Temperature
0          20.0
1          20.0
2          21.0
3          23.0
4          22.0
..      ...
103       17.0
104       18.0
105       19.0
106       20.0
107       21.0
```

```
[72 rows x 5 columns]
```

```
[37]: data_tidy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 108 entries, 0 to 107
```

```
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0    Date            108 non-null    object
1    Drug_Name        108 non-null    object
2    time             108 non-null    object
3    Pressure         95 non-null     float64
4    Temperature      95 non-null     float64
dtypes: float64(2), object(3)
memory usage: 4.3+ KB
```

```
[42]: "Rahul Janghu".split()
```

```
[42]: ['Rahul', 'Janghu']
```

```
[43]: data_tidy['Date'].str.split('-') # List of items in string dtype only
```

```
[43]: 0      [15, 10, 2020]
      1      [15, 10, 2020]
      2      [15, 10, 2020]
      3      [15, 10, 2020]
      4      [15, 10, 2020]
      ...
     103     [17, 10, 2020]
     104     [17, 10, 2020]
     105     [17, 10, 2020]
     106     [17, 10, 2020]
     107     [17, 10, 2020]
      Name: Date, Length: 108, dtype: object
```

```
[44]: def extract_year(date):
      return date[2]
```

```
[49]: data_tidy['Date'].str.split('-').apply(extract_year).astype('int')
```

```
[49]: 0      2020
      1      2020
      2      2020
      3      2020
      4      2020
      ...
     103     2020
     104     2020
     105     2020
     106     2020
     107     2020
      Name: Date, Length: 108, dtype: int64
```

```
[50]: # Better?
      # datetime methods
```

```
[52]: data_tidy.head()
```

```
[52]: Parameter      Date      Drug_Name      time  Pressure  \
0      15-10-2020  diltiazem hydrochloride  10:30:00      18.0
1      15-10-2020  diltiazem hydrochloride  11:30:00      19.0
2      15-10-2020  diltiazem hydrochloride  12:30:00      20.0
3      15-10-2020  diltiazem hydrochloride   1:30:00      12.0
4      15-10-2020  diltiazem hydrochloride   2:30:00      13.0

Parameter  Temperature
0          20.0
1          20.0
2          21.0
3          23.0
4          22.0
```

```
[54]: data_tidy['Date'] + ' ' + data_tidy['time']
```

```
[54]: 0      15-10-2020 10:30:00
1      15-10-2020 11:30:00
2      15-10-2020 12:30:00
3      15-10-2020  1:30:00
4      15-10-2020  2:30:00
...
103    17-10-2020  5:30:00
104    17-10-2020  6:30:00
105    17-10-2020  7:30:00
106    17-10-2020  8:30:00
107    17-10-2020  9:30:00
Length: 108, dtype: object
```

```
[55]: data_tidy['timestamp'] = data_tidy['Date'] + ' ' + data_tidy['time']
```

```
[56]: data_tidy.head()
```

```
[56]: Parameter      Date      Drug_Name      time  Pressure  \
0      15-10-2020  diltiazem hydrochloride  10:30:00      18.0
1      15-10-2020  diltiazem hydrochloride  11:30:00      19.0
2      15-10-2020  diltiazem hydrochloride  12:30:00      20.0
3      15-10-2020  diltiazem hydrochloride   1:30:00      12.0
4      15-10-2020  diltiazem hydrochloride   2:30:00      13.0

Parameter  Temperature      timestamp
0          20.0  15-10-2020 10:30:00
```

1	20.0	15-10-2020 11:30:00
2	21.0	15-10-2020 12:30:00
3	23.0	15-10-2020 1:30:00
4	22.0	15-10-2020 2:30:00

```
[57]: data_tidy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Date            108 non-null   object
1   Drug_Name       108 non-null   object
2   time            108 non-null   object
3   Pressure        95 non-null    float64
4   Temperature     95 non-null    float64
5   timestamp       108 non-null   object
dtypes: float64(2), object(4)
memory usage: 5.2+ KB
```

```
[58]: # to datetime format
```

```
[59]: data_tidy['timestamp'] = pd.to_datetime(data_tidy['timestamp'])
```

```
[60]: data_tidy.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 108 entries, 0 to 107
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Date            108 non-null   object
1   Drug_Name       108 non-null   object
2   time            108 non-null   object
3   Pressure        95 non-null    float64
4   Temperature     95 non-null    float64
5   timestamp       108 non-null   datetime64[ns]
dtypes: datetime64[ns](1), float64(2), object(3)
memory usage: 5.2+ KB
```

```
[ ]:
```

```
[61]: data_tidy['timestamp'].head()
```

```
[61]: 0    2020-10-15 10:30:00
      1    2020-10-15 11:30:00
```

```
2    2020-10-15 12:30:00
3    2020-10-15 01:30:00
4    2020-10-15 02:30:00
Name: timestamp, dtype: datetime64[ns]
```

```
[80]: data_tidy['timestamp'].dt.day_name()
```

```
[80]: 0    Thursday
      1    Thursday
      2    Thursday
      3    Thursday
      4    Thursday
      ...
     103   Saturday
     104   Saturday
     105   Saturday
     106   Saturday
     107   Saturday
Name: timestamp, Length: 108, dtype: object
```

```
[79]: data_tidy['timestamp'].dt.month_name()
```

```
[79]: 0    October
      1    October
      2    October
      3    October
      4    October
      ...
     103   October
     104   October
     105   October
     106   October
     107   October
Name: timestamp, Length: 108, dtype: object
```

```
[75]: data_tidy['timestamp'].dt.second
```

```
[75]: 0    0
      1    0
      2    0
      3    0
      4    0
      ..
     103    0
     104    0
     105    0
     106    0
```

```
107    0
Name: timestamp, Length: 108, dtype: int64
```

```
[68]: data_tidy['timestamp'].dt.year
```

```
[68]: 0      2020
      1      2020
      2      2020
      3      2020
      4      2020
      ...
     103     2020
     104     2020
     105     2020
     106     2020
     107     2020
Name: timestamp, Length: 108, dtype: int64
```

```
[62]: data_tidy['timestamp'].dt.hour
```

```
[62]: 0      10
      1      11
      2      12
      3       1
      4       2
      ..
     103      5
     104      6
     105      7
     106      8
     107      9
Name: timestamp, Length: 108, dtype: int64
```

```
[64]: data_tidy['timestamp'].dt.date
```

```
[64]: 0      2020-10-15
      1      2020-10-15
      2      2020-10-15
      3      2020-10-15
      4      2020-10-15
      ...
     103     2020-10-17
     104     2020-10-17
     105     2020-10-17
     106     2020-10-17
     107     2020-10-17
Name: timestamp, Length: 108, dtype: object
```

```
[65]: data_tidy['timestamp'].dt.day
```

```
[65]: 0      15
      1      15
      2      15
      3      15
      4      15
      ..
     103     17
     104     17
     105     17
     106     17
     107     17
      Name: timestamp, Length: 108, dtype: int64
```

```
[66]: data_tidy['timestamp'].dt.month
```

```
[66]: 0      10
      1      10
      2      10
      3      10
      4      10
      ..
     103     10
     104     10
     105     10
     106     10
     107     10
      Name: timestamp, Length: 108, dtype: int64
```

```
[81]: # HW: Read datetime documentation
```

```
[82]: movies = pd.read_csv('movies.csv', index_col = 0)
      directors = pd.read_csv('directors.csv', index_col = 0)
```

```
[84]: data = movies.merge(directors, how='left', left_on = 'director_id',
      ↪right_on='id')
```

```
[88]: data.drop(['director_id', 'id_y'], axis = 1, inplace = True)
```

```
[90]: data.head()
```

```
[90]:   id_x  budget  popularity  revenue \
0  43597  237000000         150  2787965087
1  43598  300000000         139   961000000
2  43599  245000000         107   880674609
3  43600  250000000         112  1084939099
```

```
4 43602 258000000 115 890871626
```

	title	vote_average	vote_count	year	\
0	Avatar	7.2	11800	2009	
1	Pirates of the Caribbean: At World's End	6.9	4500	2007	
2	Spectre	6.3	4466	2015	
3	The Dark Knight Rises	7.6	9106	2012	
4	Spider-Man 3	5.9	3576	2007	

	month	day	director_name	gender
0	Dec	Thursday	James Cameron	Male
1	May	Saturday	Gore Verbinski	Male
2	Oct	Monday	Sam Mendes	Male
3	Jul	Monday	Christopher Nolan	Male
4	May	Tuesday	Sam Raimi	Male

```
[93]: # Question 1: What is the total span of years covered by the movies dataset,
# calculated from its earliest to its latest release year?
```

```
[99]: no_of_years = data['year'].max() - data['year'].min()
print(no_of_years)
```

```
40
```

- Question 2: Using the movies data set, get release date in Day, Month, Year format.
- Example Date format: Thursday-Dec-2009

```
[103]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 1465 entries, 0 to 1464
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id_x             1465 non-null   int64
1   budget           1465 non-null   int64
2   popularity        1465 non-null   int64
3   revenue           1465 non-null   int64
4   title            1465 non-null   object
5   vote_average      1465 non-null   float64
6   vote_count        1465 non-null   int64
7   year             1465 non-null   int64
8   month            1465 non-null   object
9   day              1465 non-null   object
10  director_name     1465 non-null   object
11  gender            1341 non-null   object
dtypes: float64(1), int64(6), object(5)
memory usage: 181.1+ KB
```

```
[107]: data.head()
```

```
[107]:    id_x    budget  popularity    revenue \
0  43597  237000000         150  2787965087
1  43598  300000000         139   961000000
2  43599  245000000         107   880674609
3  43600  250000000         112  1084939099
4  43602  258000000         115   890871626

        title  vote_average  vote_count  year \
0          Avatar           7.2       11800  2009
1  Pirates of the Caribbean: At World's End           6.9        4500  2007
2              Spectre           6.3       4466  2015
3      The Dark Knight Rises           7.6       9106  2012
4      Spider-Man 3           5.9       3576  2007

    month    day    director_name  gender
0    Dec  Thursday      James Cameron   Male
1    May  Saturday      Gore Verbinski   Male
2    Oct   Monday        Sam Mendes   Male
3    Jul   Monday  Christopher Nolan   Male
4    May  Tuesday        Sam Raimi    Male
```

```
[112]: data['day'] + '-' + data['month'] + '-' + data['year'].astype('str')
```

```
[112]: 0    Thursday-Dec-2009
1    Saturday-May-2007
2    Monday-Oct-2015
3    Monday-Jul-2012
4    Tuesday-May-2007
...
1460    Monday-May-1978
1461    Tuesday-Sep-1994
1462    Friday-Aug-2009
1463    Friday-Jul-1990
1464    Friday-Sep-1992
Length: 1465, dtype: object
```

```
[113]: data['release_date'] = data['day'] + '-' + data['month'] + '-' + data['year'].
      ↪astype('str')
```

```
[114]: data.head()
```

```
[114]:    id_x    budget  popularity    revenue \
0  43597  237000000         150  2787965087
1  43598  300000000         139   961000000
2  43599  245000000         107   880674609
```

```

3  43600  250000000      112  1084939099
4  43602  258000000      115   890871626

```

```

              title  vote_average  vote_count  year  \
0              Avatar           7.2       11800  2009
1  Pirates of the Caribbean: At World's End           6.9       4500  2007
2              Spectre           6.3       4466  2015
3      The Dark Knight Rises           7.6       9106  2012
4      Spider-Man 3           5.9       3576  2007

```

```

    month    day    director_name  gender    release_date
0   Dec  Thursday    James Cameron   Male  Thursday-Dec-2009
1   May  Saturday    Gore Verbinski   Male  Saturday-May-2007
2   Oct   Monday      Sam Mendes   Male   Monday-Oct-2015
3   Jul   Monday  Christopher Nolan   Male   Monday-Jul-2012
4   May   Tuesday      Sam Raimi   Male   Tuesday-May-2007

```

```
[115]: # Question 3: What is the % of male directors in the given data?
```

```
[127]: data['gender'].value_counts(normalize=True) * 100
```

```

[127]: Male      97.613721
      Female     2.386279
      Name: gender, dtype: float64

```

```
[ ]:
```

```
[129]: # Question 4: How many different movies have been directed in the year 2010?
```

```
[133]: len(data[data['year'] == 2010]['title'])
```

```
[133]: 63
```

```

[134]: # Question 5: How many different directors have directed a movie since 2010_
      ↪ (inclusive)?

```

```
[139]: data[data['year'] >= 2010]['director_name'].nunique()
```

```
[139]: 163
```

```

[140]: # Question 6: How would you correctly retrieve the names of all distinct
      # directors who have directed movies since 2010 (inclusive)?

```

```
[142]: # data[data['year'] >= 2010]['director_name'].unique()
```

```
[ ]:
```

```
[141]: # Question 7: Display the name of the director who directed the Spider-Man 3 movie.
```

```
[147]: data[data['title'] == 'Spider-Man 3']['director_name']
```

```
[147]: 4    Sam Raimi
      Name: director_name, dtype: object
```

```
[ ]:
```

```
[149]: # Question 8: How many different movies have been directed by James Cameron
      # between year 2005 to 2010?
```

```
[152]: data[(data['director_name'] == 'James Cameron') & (data['year'] >= 2005) &
      (data['year'] <= 2010)]
```

```
[152]:    id_x    budget  popularity    revenue  title  vote_average  vote_count \
0  43597  237000000         150  2787965087  Avatar           7.2         11800

    year month    day  director_name  gender    release_date
0  2009   Dec  Thursday   James Cameron   Male  Thursday-Dec-2009
```

```
[154]: data[(data['director_name'] == 'James Cameron') & (data['year'] >= 2005) &
      (data['year'] <= 2010)]['title']
```

```
[154]: 1
```

```
[156]: # Question 9: Among the top 10 most popular movies, which is the movie with
      # the 2nd highest vote_avg?
```

```
[163]: top_10_popular = data.sort_values(by = 'popularity', ascending = False).head(10)
```

```
[167]: top_10_popular.sort_values(by = 'vote_average', ascending=False).
      iloc[1]['title']
```

```
[167]: 'The Dark Knight'
```

```
[168]: # data.sort_values(by = ['popularity', 'vote_average'], ascending = [False,
      False]).iloc[1]
```

```
[ ]:
```

```
[170]: #question : Question 10:
      #How many movies that are among top 10 popular movies are also among top 10
      rated (vote_avg)?
```

```
[174]: top_10_popular_name = data.sort_values(by='popularity', ascending=False).  
      ↪head(10)['title']
```

```
[175]: top_10_popular_name
```

```
[175]: 58                Interstellar  
      78                Mad Max: Fury Road  
      119    Pirates of the Caribbean: The Curse of the Bla...  
      120                The Hunger Games: Mockingjay - Part 1  
      45                The Dark Knight  
      162                The Martian  
      59                Inception  
      5                Batman v Superman: Dawn of Justice  
      0                Avatar  
      383                Fight Club  
      Name: title, dtype: object
```

```
[176]: top_10_rated_name = data.sort_values(by='vote_count', ascending=False).  
      ↪head(10)['title']
```

```
[177]: top_10_rated_name
```

```
[177]: 59                Inception  
      45                The Dark Knight  
      0                Avatar  
      58                Interstellar  
      178    Django Unchained  
      78                Mad Max: Fury Road  
      383                Fight Club  
      3                The Dark Knight Rises  
      364                The Matrix  
      46                Iron Man  
      Name: title, dtype: object
```

```
[179]: len(data[data['title'].isin(top_10_popular_name) & data['title'].  
      ↪isin(top_10_rated_name)])
```

```
[179]: 6
```

```
[182]: #question : In which year did Christopher Nolan produced most number of movies?
```

```
[185]: chris_movies = data[data['director_name'] == 'Christopher Nolan']
```

```
[189]: chris_movies['year'].value_counts()
```

```
[189]: 2012    1  
      2008    1
```

```

2014    1
2010    1
2005    1
2002    1
2006    1
2000    1
Name: year, dtype: int64

```

```
[188]: chris_movies.groupby('year')['title'].count()
```

```

[188]: year
2000    1
2002    1
2005    1
2006    1
2008    1
2010    1
2012    1
2014    1
Name: title, dtype: int64

```

```
[ ]:
```

```
[191]: # In which decade, the most number of movies were produced?
```

```
[192]: data['year'].min(), data['year'].max()
```

```
[192]: (1976, 2016)
```

```

[199]: bins = [1970, 1980, 1990, 2000, 2010, 2020]
       labels = ['1970', '1980', '1990', '2000', '2010']

```

```
[200]: data['year_bucket'] = pd.cut(data['year'], bins=bins, labels=labels)
```

```
[201]: data.head()
```

```

[201]:
   id_x  budget  popularity  revenue \
0  43597  237000000         150  2787965087
1  43598  300000000         139   961000000
2  43599  245000000         107   880674609
3  43600  250000000         112  1084939099
4  43602  258000000         115   890871626

   title  vote_average  vote_count  year \
0  Avatar             7.2        11800  2009
1  Pirates of the Caribbean: At World's End  6.9         4500  2007
2  Spectre             6.3         4466  2015

```

3		The Dark Knight Rises	7.6	9106	2012
4		Spider-Man 3	5.9	3576	2007

	month	day	director_name	gender	release_date	year_bucket
0	Dec	Thursday	James Cameron	Male	Thursday-Dec-2009	2000
1	May	Saturday	Gore Verbinski	Male	Saturday-May-2007	2000
2	Oct	Monday	Sam Mendes	Male	Monday-Oct-2015	2010
3	Jul	Monday	Christopher Nolan	Male	Monday-Jul-2012	2010
4	May	Tuesday	Sam Raimi	Male	Tuesday-May-2007	2000

```
[204]: data['year_bucket'].value_counts()
```

```
[204]: 2000    644
      1990    376
      2010    293
      1980    126
      1970     26
      Name: year_bucket, dtype: int64
```

```
[205]: data['year_bucket'].value_counts()[0]
```

```
[205]: 644
```

1 HW Questions

- 1. Within the decade with most number of movies, which were the top 5 contributing directors (who produced most)?
- 2. Display movie details of only those years where number of movies produced are more than 70.
- 3. Create a new metric called revenue_per_budget i.e. revenue/budget and round it off up to 2 decimal places.

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

[]: